Project - Hardware Design For AI

MTech-AI

B04 TPU Simulator using Systolic Array Architecture

Submitted by

Team Members:

1. Prasun Kumar Tripathi (M25AI2151)
2. Kamal Kumar (M25AI2148)
3. Vikash Kumar (M25AI2166)
4. Sachin Yadav (M25AI2051)

Master Of Technology - AI

Indian Institute of Technology, Jodhpur

Computer Science

April, 2026

Abstract

This project presents a pure Python simulator for Tensor Processing Unit (TPU) style matrix multiplication using a systolic-array architecture. The project demonstrates Processing Elements, multiply-accumulate operations, matrix generation, correctness validation, performance benchmarking, hardware comparison, and an executable Colab notebook.

The executed notebook confirms that the main simulator result matches NumPy, the automated test suite passes all 6 test categories, and the 50 x 50 benchmark records NumPy CPU time of 0.000117 seconds and Python TPU simulator time of 0.081418 seconds.

How to Run and Verify This Project

This section is for the evaluator who wants to reproduce the output independently. The recommended route is Google Colab because it opens directly in the browser and executes the same GitHub code.

GitHub Repository: https://github.com/prasun-iitj/HDAI_Course-project_TPU-Simulator

Direct Colab Notebook: https://colab.research.google.com/github/prasun-iitj/HDAI_Course-project_TPU-Simulator/blob/main/TPU_Simulator_Colab.ipynb

Recommended Verification Method: Google Colab

- Open the direct Colab notebook link above.
- Click Runtime > Run all.
- The notebook clones the GitHub repository, installs dependencies, and runs the simulator scripts.
- Expected result: main simulator prints that results match NumPy, and test_simulator.py prints TOTAL: 6/6 tests passed.

Local Verification Method

Run the following commands locally:

Command sequence
git clone https://github.com/prasun-iitj/HDAI_Course-project_TPU-Simulator.git
cd HDAI_Course-project_TPU-Simulator
pip install numpy matplotlib seaborn
python demo_complete.py

Smaller targeted checks: python tpu_simulator.py, python test_simulator.py, python benchmark.py, and python hardware_comparison.py.

Table of Contents

1. INTRODUCTION
2. LITERATURE REVIEW
3. METHODOLOGY
4. EXPERIMENTAL SETUP
5. RESULTS AND DISCUSSION
6. CONCLUSIONS
References
Appendix A: Source Code Listing

List of Figures

Figure 1: Conceptual systolic-array dataflow diagram
Figure 2: Sample matrix heatmaps and result matrix
Figure 3: Theoretical cycle speedup chart
Figure 4: Observed execution timing chart

1. INTRODUCTION

1.1 Background and Motivation

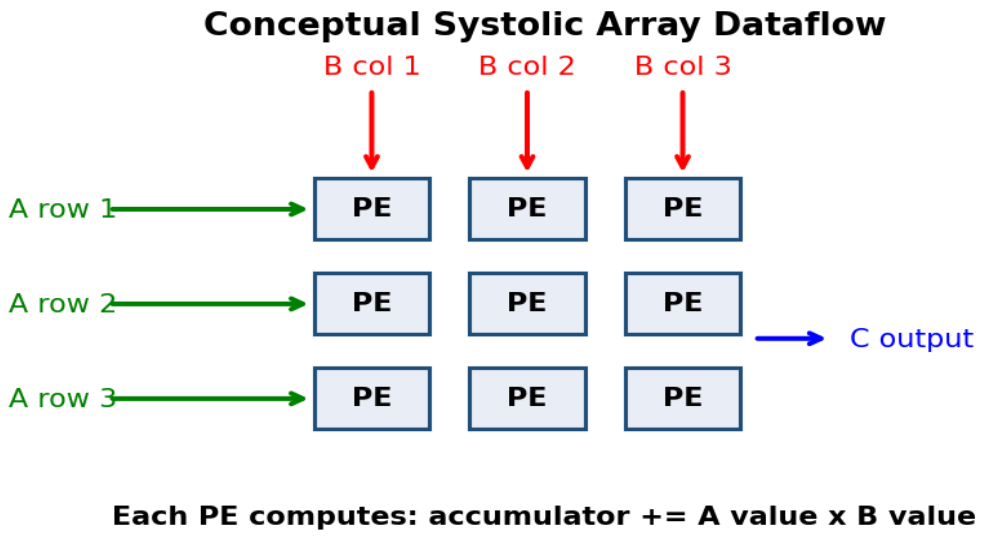

Figure 1: Conceptual systolic-array dataflow through Processing Elements.

Modern AI systems are dominated by matrix operations. Neural network layers, attention projections, embeddings, and convolution lowering all rely on dense linear algebra. TPUs address this need with domain-specific hardware focused on tensor operations.

A systolic array is a regular grid of simple Processing Elements where matrix values flow through the array and each PE performs multiply-accumulate work.

1.2 Problem Statement

The project asks how TPU-style matrix multiplication can be demonstrated, validated, and explained without special hardware.

1.3 Objectives

- Implement a TPU simulator around a systolic-array abstraction.
- Generate test matrices and multiply them through the simulator.
- Validate all results against numpy.dot.
- Run automated tests for normal, special, and invalid cases.
- Measure CPU versus simulator wall-clock time.
- Explain why real TPUs gain from parallel systolic execution.

2. LITERATURE REVIEW

2.1 TPU Architecture and Systolic Arrays

TPUs are specialised accelerators designed for neural-network tensor computation. They allocate large datapaths to regular arithmetic arrays and local data movement.

2.2 Matrix Multiplication for AI Workloads

For n x n matrices, traditional sequential multiplication performs O(n^3) scalar operations. A simplified systolic model estimates approximately 2n - 1 wavefront cycles.

2.3 Related Work

Kung introduced systolic architectures as a foundation for regular parallel computing. Jouppi and colleagues analysed Google TPUs in datacenter workloads.

3. METHODOLOGY

3.1 Simulator Architecture

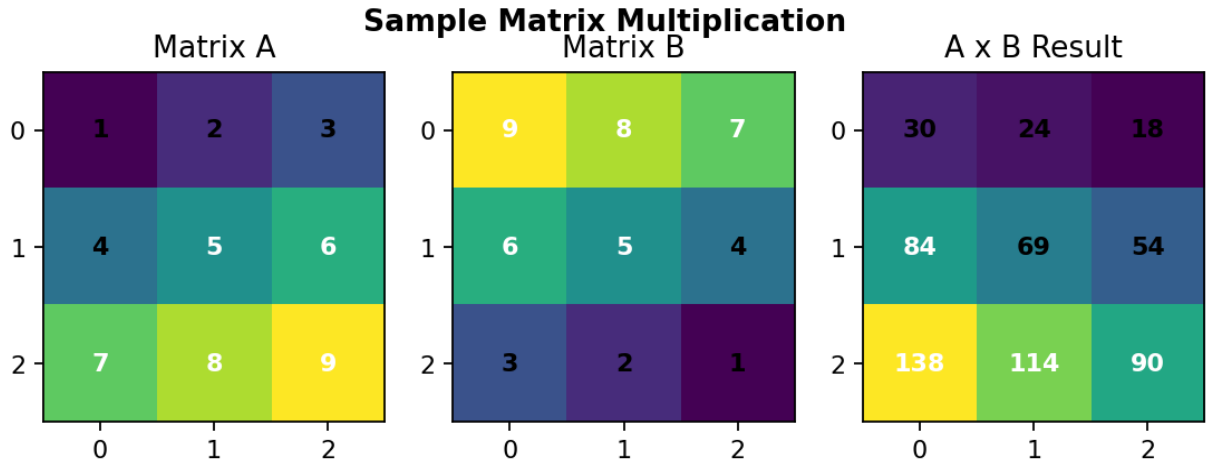

Figure 2: Sample input matrices and the resulting product matrix.

The repository is organised as small modules for processing elements, systolic array multiplication, matrix generation, simulation, benchmarking, testing, and hardware comparison.

Module	Purpose
processing_element.py	Single PE multiply-accumulate abstraction
systolic_array.py	Main simulator class and multiplication logic
matrix_generator.py	Random matrix generation
test_simulator.py	Six-category test suite
TPU_Simulator_Colab.ipynb	Executable notebook with saved outputs

3.2 Validation Strategy

Each simulated result is compared with numpy.dot. Tests include small, medium, identity, zero, larger, and invalid-input cases.

4. EXPERIMENTAL SETUP

The project runs locally on Windows and in Google Colab. No GPU, CUDA, TPU runtime, or special hardware is required.

Component	Role
Python 3.13.12	Local script and notebook execution
NumPy 2.4.4	Reference matrix multiplication
Matplotlib / Seaborn	Visualisation and report figures
GitHub / Colab	Versioned and browser-based verification

5. RESULTS AND DISCUSSION

5.1 Correctness Validation

The main simulator printed: VERIFICATION SUCCESSFUL - Results match. The full automated test suite printed TOTAL: 6/6 tests passed and ALL TESTS PASSED.

Validation Item	Observed Result
Main simulator	Matched NumPy
Small 2 x 2	Passed
Medium 4 x 4	Passed
Identity matrix	Passed
Zero matrix	Passed
Larger 8 x 8	Passed
Error handling	Passed
Overall	6/6 tests passed

5.2 Execution Time Benchmark

Matrix Size	NumPy CPU	TPU Simulator	Observation
50 x 50	0.000117 s	0.081418 s	CPU 695.5x faster

5.3 Cycle Reduction Analysis

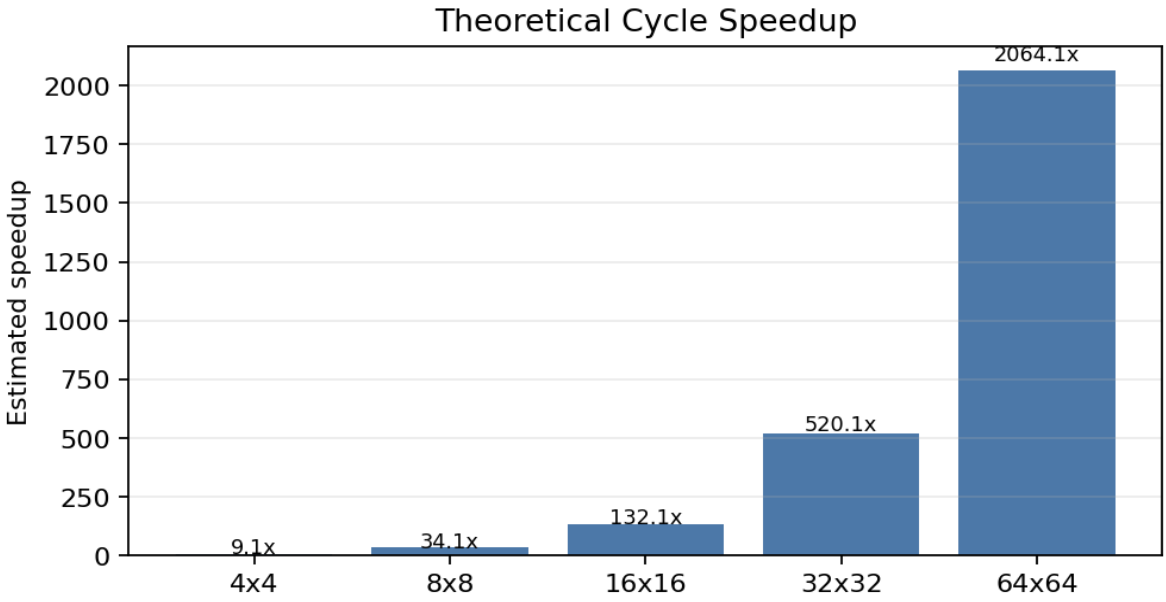

Figure 3: Theoretical cycle reduction as matrix size increases.

Matrix Size	CPU Cycles	TPU Cycles	Estimated Speedup
4 x 4	64	7	9.1x
8 x 8	512	15	34.1x
16 x 16	4,096	31	132.1x
32 x 32	32,768	63	520.1x
64 x 64	262,144	127	2064.1x

5.4 Observed Timing Across Matrix Sizes

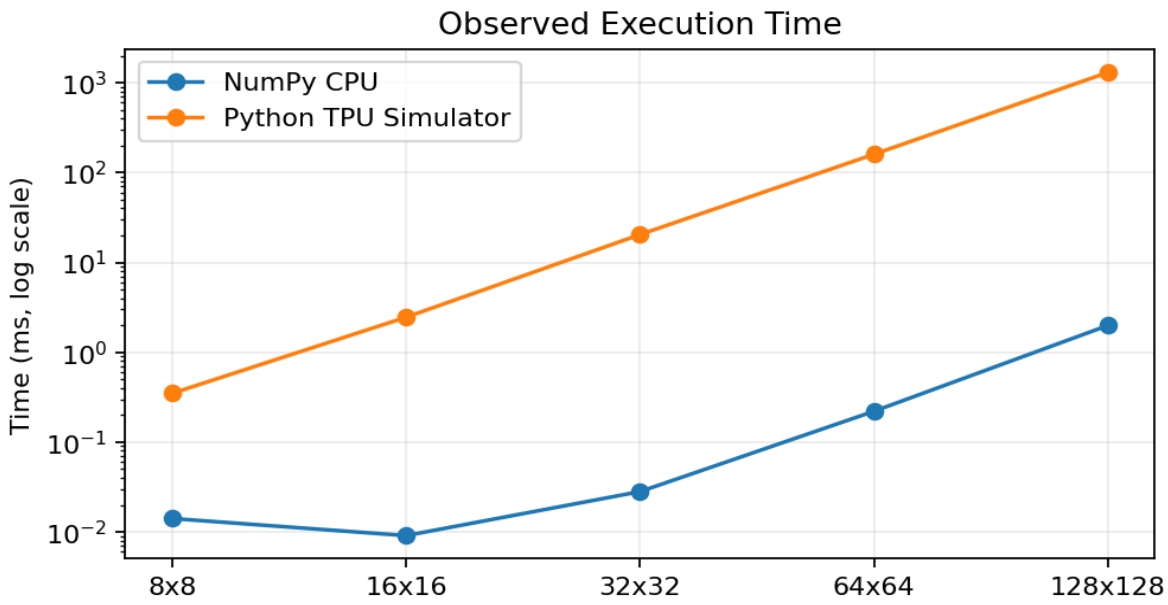

Figure 4: Observed CPU and simulator timing from the executed notebook.

Matrix Size	NumPy CPU	Python TPU Sim	Ratio
8 x 8	0.0141 ms	0.3486 ms	0.04x
16 x 16	0.0091 ms	2.4543 ms	0.00x
32 x 32	0.0281 ms	20.4291 ms	0.00x
64 x 64	0.2205 ms	160.0144 ms	0.00x
128 x 128	1.9898 ms	1305.2289 ms	0.00x

5.5 Hardware Comparison

Hardware	Time	Power	Performance	Efficiency
CPU	~10 sec	125 W	10 GFLOPS	0.08 GFLOPS/W
GPU	~0.5 sec	250 W	200 GFLOPS	0.8 GFLOPS/W
TPU	~0.1 sec	40 W	500 GFLOPS	12.5 GFLOPS/W

At the software level, matrix multiplication is correct and fully tested. At the architecture level, the cycle model and hardware comparison explain why real systolic arrays can achieve high throughput and energy efficiency.

6. CONCLUSIONS

The TPU Simulator project successfully demonstrates matrix multiplication through a systolic-array-inspired architecture. It includes a PE abstraction, simulator class, generated workloads, validation scripts, benchmark scripts, hardware comparison analysis, and a Colab notebook with saved outputs.

Future improvements include implementing full cycle-by-cycle systolic dataflow, supporting rectangular matrices, expanding visualisations, and adding richer profiling metrics.

References

[1] H. T. Kung, "Why Systolic Architectures?", Computer, 1982.

[2] N. P. Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit", ISCA, 2017.

[3] N. P. Jouppi et al., "A Domain-Specific Supercomputer for Training Deep Neural Networks", Communications of the ACM, 2020.

[4] Google Cloud TPU Documentation.

[5] NumPy Documentation.

[6] Project repository documentation: README.md, DESIGN.md, INSTRUCTIONS.md, QUICKSTART.md, TPU_Project_Report.md.

Appendix A: Source Code Listing

A.1 Main Execution Commands

python tpu_simulator.py
python benchmark.py
python test_simulator.py
python hardware_comparison.py
python demo_complete.py

A.2 Core Multiplication Logic

```
for i in range(n):
    for j in range(n):
        for k in range(n):
            c[i][j] += a[i][k] * b[k][j]
```